

App Dev Project Report

1. Student Details

Name: Debopam Chakraborty

Roll Number: 22F1000777

Email: 22f1000777@ds.study.iitm.ac.in

About Me: I am a student of IIT Madras BS in Data Science and Applications program, finishing diploma and entering degree level soon. I have deep interest in Data Science and app development.

2. Project Details

Project Title: Placement Portal

Problem Statement:

To design and develop a web-based Placement Portal that streamlines the campus recruitment process by connecting students, companies, and administrators through a centralized platform. The system enables companies to publish placement drives, students to apply for opportunities, and administrators to manage users and monitor placement activities efficiently.

Approach:

The application follows a client-server architecture consisting of a Flask-based REST API backend and a Vue.js Single Page Application frontend. SQLAlchemy is used for database interaction with SQLite, while Flask-Login provides session-based authentication. Celery and Redis are used for asynchronous background tasks such as reminder emails, monthly placement reports, and CSV exports. The user interface is developed using Bootstrap to ensure a responsive and intuitive experience across devices.

3. AI/LLM Declaration

I used ChatGPT (GPT-5.5) to assist in writing SQLAlchemy model definitions, creating API documentation (yaml file), creating ER diagram, creating this document, writing unit tests for backend, debugging problems and improving variable naming consistency. The extent of AI/LLM usage is around 10–15%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend REST API framework
SQLAlchemy	Object Relational Mapper (ORM) for SQLite
SQLite	Lightweight relational database
Vue.js 3	Single Page Application frontend
Vue Router	Client-side routing
Pinia	Frontend state management
Bootstrap 5	Responsive frontend styling
Flask-Login	Authentication and session management
Flask-Mail	Sending email notifications
Jinja2	HTML template rendering for emails
Celery	Background task processing
Redis	Celery broker and result backend
Pytest	Backend unit testing
Vite	Frontend development server and build tool

5. Database Schema / ER Diagram

Tables:

1. **User** — stores authentication details such as email, password hash, and user role.
2. **Student** — stores student profile information, resume details, and blacklist status.
3. **Company** — stores company profile details and approval status.
4. **PlacementDrive** — stores placement opportunities created by companies.
5. **JobApplication** — stores student applications and recruitment status.

Enum Values

- `USER.role`: admin, company, student
- `COMPANY.approval_status`: pending, approved, rejected, blacklisted
- `PLACEMENT_DRIVE.status`: pending, active, declined, closed
- `JOB_APPLICATION.status`: applied, shortlisted, selected, rejected

Relationship Notes

- `student.user_id` is unique, so each student profile belongs to exactly one user.
- `company.user_id` is unique, so each company profile belongs to exactly one user.
- Deleting a user cascades to the linked student or company profile.
- Deleting a company cascades to its placement drives.
- Deleting a placement drive cascades to its job applications.

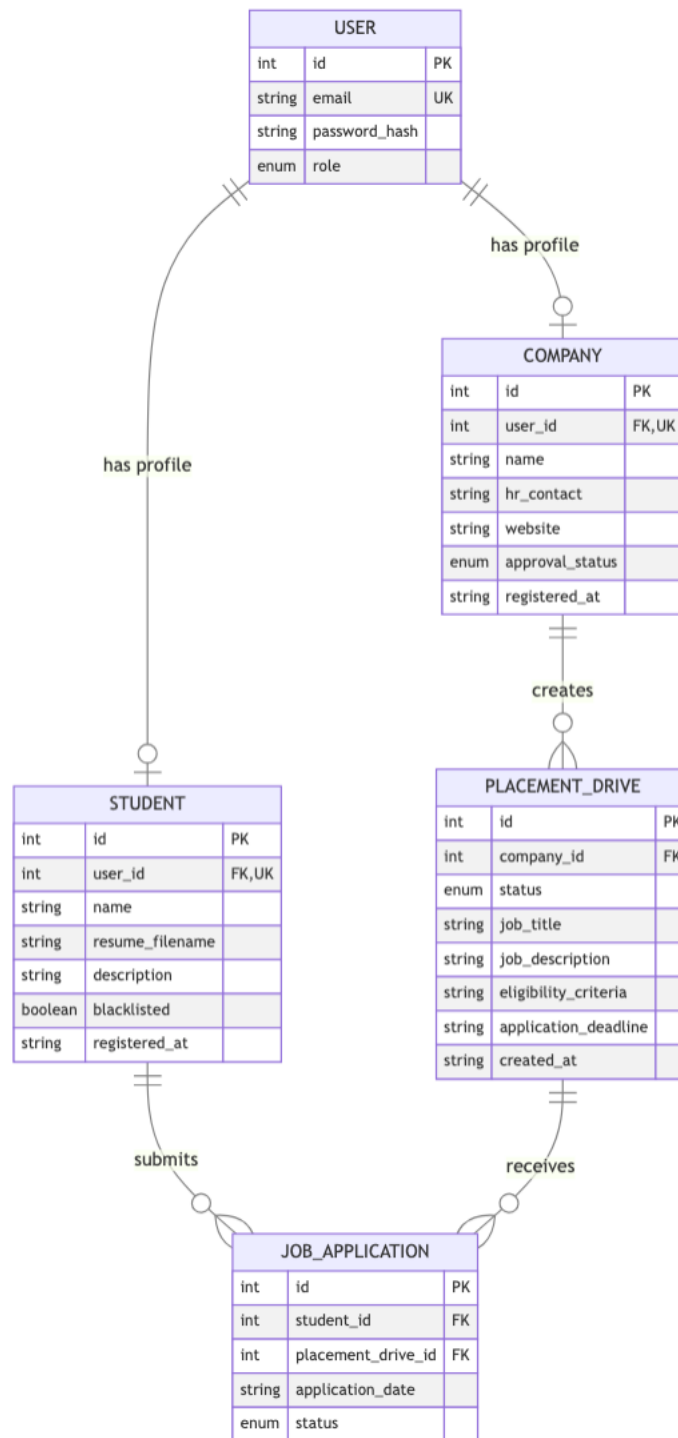


Figure 1: ERDiagram
3

6. API Resource Endpoints

Authentication

Endpoint	Method	Description
/api/auth/login	POST	Authenticate user
/api/auth/logout	POST	Logout current user
/api/auth/me	GET	Fetch authenticated user
/api/auth/register/student	POST	Register a student
/api/auth/register/company	POST	Register a company

Student

Endpoint	Method	Description
/api/student/profile	GET	View own profile
/api/student/profile	PATCH	Update own profile
/api/student/placement-drives	GET	View available placement drives
/api/student/companies	GET	View approved companies
/api/student/applications	POST	Apply to a placement drive
/api/student/export	POST	Export application history as CSV
/api/student/resume	GET	Download resume
/api/student/resume	POST	Upload resume

Company

Endpoint	Method	Description
/api/company/profile	GET	View own profile
/api/company/profile	PATCH	Update own profile
/api/company/placement-drives	GET	View company placement drives
/api/company/placement-drives	POST	Create placement drive
/api/company/placement-drives/{id}	PATCH	Edit placement drive
/api/company/applications	GET	View received applications
/api/company/applications/{id}	PATCH	Update application status

Administrator

Endpoint	Method	Description
/api/admin/companies	GET	View all companies
/api/admin/companies/{id}	PATCH	Approve / reject company
/api/admin/companies/{id}	DELETE	Delete company
/api/admin/students	GET	View all students
/api/admin/students/{id}	PATCH	Blacklist / unblacklist student
/api/admin/placement-drives	GET	View all placement drives

YAML API Definition File: Included separately in the submission ZIP as `api.yaml`.

7. Architecture and Features

Architecture Overview:

- **Frontend** – Vue.js Single Page Application using Vue Router and Pinia.
- **Backend** – Flask application using the Application Factory pattern with modular Blueprints.
- **Models** – SQLAlchemy ORM models.
- **Services** – Business logic for reports, reminders, exports, and email notifications.
- **Tasks** – Celery workers executing asynchronous and scheduled background jobs.
- **Templates** – Jinja2 HTML email templates.
- **Redis** – Message broker and result backend for Celery.

Implemented Features:

- Student, Company, and Administrator authentication
- Student and Company registration
- Company approval workflow
- Student and Company profile management
- Resume upload and download
- Placement drive creation, editing, approval, reopening, and closure
- Student application management
- Applicant shortlisting, selection, and rejection
- Student blacklist management
- Responsive Bootstrap-based user interface
- Role-based authorization

Additional Features:

- Daily reminder emails for approaching application deadlines
- Monthly administrator placement reports

- Asynchronous CSV export of student application history
- HTML email templates using Jinja2
- RESTful API design
- Session-based authentication using Flask-Login
- Unit testing using Pytest

8. Video Presentation

Drive Link: https://drive.google.com/file/d/15zZVJpv_IXYpf2eF5UAqMYWTfpXfgOUd/view?usp=sharing